Lapor Aman

Confidential Reporting of Violence on Campus

Frenaldy Bestabba Hasugian (NRP 5025241156) · I Made Ryan Darmajaya (NRP 5025241159) · Muh. Aqil Alqadri Syahid (NRP 5025241161) · Kadek Andra Wikanjaya Putra (NRP 5025241187)

Information Security / IUP — Assignment 1 · Repository: https://github.com/andra-wp/testing

1. Scenario: Reporting Violence on Campus

Students who experience or witness violence on campus — sexual violence and harassment, gender-based violence online (e.g. spreading intimate images), bullying, cyberbullying, physical or psychological violence — often stay silent because they fear that their identity and their story will spread beyond the people handling it. Lapor Aman gives them one channel to the campus task force (**Satgas**) that handles such cases. A **reporter** registers, verifies their identity once with a KTP scan, files a report (category, chronology, location or link, perpetrator) with documents and videos, and follows its status. An **officer** (accounts are created only by an administrator script) works through a queue, sees the reporter's identity masked, and may **preview** — never download — only the evidence the reporter shared. E-mailing evidence to a staff member would leave readable copies in several mailboxes with no record of who opened them; here everything personal is encrypted before it is stored, only the owner can download it, and every access is logged and visible to the reporter.

1.1. Which data is private

Data	What it contains here	Private?	Basis (GDPR / UU PDP)
Identity	full name, NIK, phone, e-mail	Yes	GDPR Art. 4(1); UU PDP Pasal 4(3)
KTP scan	NIK, photo, address, date of birth	Yes	UU PDP Pasal 4(3); the photo is only compared by eye, not processed for biometric identification (GDPR Recital 51)
Report text, file names, messages	chronology, place or link, perpetrator and witnesses	Yes; specific for sexual violence	GDPR Art. 9(1) (sex life); perpetrator and witnesses are data subjects too, Art. 4(1)
Office documents	written statements, medical/visum reports	Yes, specific	health data: GDPR Art. 9(1); UU PDP Pasal 4(2)(a)
Screenshots, video	messages, faces, voices; intimate images (KBGO)	Yes; specific if intimate	GDPR Art. 4(1), 9(1)

Table 1: Classification of stored data. Treated as **not** private and kept in plaintext: record IDs, status, category, timestamps and file sizes, which the queue and statistics need (but see §4 on the category). GDPR Art. 5(1)(f) and 32(1)(a) name confidentiality, integrity and encryption as required safeguards.

2. Design and Justification

2.1. Architecture

A React frontend talks to a FastAPI backend; SQLite holds metadata, wrapped keys and the ciphertext of short text fields, while files are streamed to disk in 1 MiB chunks (one directory per cipher), so a 200 MB video is never held in memory. Encryption runs on the server, not in the browser, because officers must preview shared evidence **through** the server's access checks and watermark, and browsers' WebCrypto offers AES but neither DES nor RC4. The cost is that the server sees plaintext during a request, so it must be trusted and served over HTTPS (§4).

2.2. Ciphers and modes of operation

The assignment asks that everything stored be protected with each of AES, DES and RC4, so every item is encrypted **three times**, once per scheme, each with its own random key; all reads use the AES copy. ECB is never used: equal plaintext blocks would give equal ciphertext blocks and reveal file structure.

AES-256-GCM. An AEAD mode: confidentiality and a 128-bit tag in one pass, accelerated by AES-NI. Its weakness, nonce reuse, cannot occur: every item has a fresh key and a random 96-bit nonce.

DES-CBC + HMAC-SHA256. DES has no authenticated mode (its 64-bit block rules out GCM), so we use CBC with a random IV and PKCS#7 padding, wrapped in Encrypt-then-MAC over IV, ciphertext and context. The MAC is checked first, which removes padding-oracle and bit-flipping attacks. Per-item keys keep every key far below the 64-bit-block (Sweet32) collision bound of ≈ 32 GiB.

RC4-drop[3072] + HMAC-SHA256. RC4 has no IV, so keystream reuse is prevented only by a fresh 128-bit key per item. The first 3072 keystream bytes, which are measurably biased, are discarded (§3), and Encrypt-then-MAC removes RC4′s malleability.

2.3. Key handling

Per item and cipher, a random key (`os.urandom`) is **wrapped** with a `MASTER_KEY` using AES-256-GCM and stored next to the ciphertext (envelope encryption). We chose this over keys derived from the reporter's password because an officer must be able to preview evidence the reporter shared without knowing that password, and because deleting the wrapped keys erases an account irrecoverably (crypto-shredding, the right to erasure). Owner, object type, ID, field and cipher are bound to every ciphertext as associated data, so a ciphertext or key moved to another row or user fails verification. Two further keys are separate from the master key: `INTEGRITY_KEY` for an HMAC of the plaintext and `BLIND_INDEX_KEY` for looking up the (encrypted) e-mail at login. Account passwords are hashed with bcrypt (cost 12).

2.4. Storage

Encrypted: identity fields, KTP scan, report text, file names, messages and all evidence. Plaintext: only the metadata in Table 1. Uploads are checked by magic bytes, not only by extension, so a disguised executable is rejected.

2.5. Authentication, access control and integrity

Login issues a short-lived JWT (HS256) kept in the browser's session storage. Downloads are allowed only to the owner; anyone else receives 404, so the file's existence is not revealed. Officers see masked identities and a watermarked, download-free preview of files the reporter shared — per file, revocable, and only while the report is open; for other files they must request access with a reason the reporter approves or denies. The KTP is visible to officers only while its verification is pending. On every read the

server unwraps the key, verifies the GCM tag or HMAC, decrypts, and compares the plaintext HMAC with the one recorded at upload; only if all match is data released, so it comes back intact or not at all. The **Check 3 ciphers** button decrypts all copies and shows that each equals the original.

3. Comparative Analysis of AES, DES and RC4

3.1. What we measure and why

A reporter waits for an upload, an owner for a download, and an attacker targets the weakest copy, so we measure: (1) time per data type of the scenario — a NIK field, a chronology, a KTP scan, an office document, a video — using exactly the application's schemes; (2) the pure ciphers versus HMAC, to separate the price of integrity; (3) fixed cost per item, which dominates small fields; (4) tampering, to test "comes back intact"; (5) measurable weaknesses: RC4 bias and the DES key and block size.

3.2. Methodology

Script `backend/benchmark/run_benchmark.py` on x86_64 with 2 cores and AES-NI, Python 3.13.17, PyCryptodome 4.2.0. Random data, chunks prepared outside the timed region, warm-up, garbage collection disabled, small payloads repeated so that each sample lasts ≥ 20 ms, optional pinning to one core; we report the median (with quartiles and coefficient of variation in the full results).

3.3. Results

Data	Size	AES-256-GCM	DES-CBC-HMAC	RC4-HMAC
NIK field	16 B	122 µs	93.8 µs	91.2 µs
Chronology	2 KB	124 µs	214 µs	104 µs
KTP scan	600 KB	815 µs	32.6 ms	3.85 ms
Office document	3 MB	3.82 ms	158 ms	17.6 ms
Video evidence	64 MB	82.7 ms	3.37 s	399 ms

Table 2: Median encryption time per scheme, incl. key wrap and tag/HMAC. Pure-cipher throughput (16 MB): AES-GCM 881, DES-CBC 19.9, RC4 325, HMAC-SHA256 372 MB/s.

3.4. Interpretation

Small fields: every scheme needs about 0.1–0.2 ms, and for the NIK field AES is even the slowest, because the fixed cost — wrapping the key (62 µs) and setting up GCM for a new key — outweighs the cipher; no user notices this. **Files:** for a 64 MB video DES is 40.7 × and RC4 4.82 × slower than AES. Because all three copies are written, processing a 200 MB video takes ≈ 12.6 s, 84 % of it caused by DES; with AES alone it would be under one second. RC4′s classic speed advantage disappears once integrity is required: pure RC4 reaches 325 MB/s, with HMAC ≈ 174 MB/s, while AES-GCM delivers 881 MB/s with its tag included. **Integrity:** flipping one random ciphertext bit 2000 times was detected every time by all three schemes, whereas without a MAC CTR and RC4 detected 0 and CBC only 5 an attacker could silently alter a date or name in a chronology, which would destroy its value as evidence. **Weaknesses:** without the drop, RC4′s second keystream byte is 0 with probability 0.74 % instead of 0.39 % ($z = 28.6$); after dropping 3072 bytes the bias is no longer measurable ($z = -1.41$). DES's 56-bit key was broken in about 56 hours by dedicated hardware in 1998. Since a thief only needs the weakest copy, **confidentiality is only as strong as DES**. We therefore recommend AES-256-GCM as the only stored copy in a real deployment; DES and RC4 are kept here because the assignment requires them.

4. Limitations and Conclusion

Storing a DES copy weakens confidentiality to 56 bits and triples disk use. `MASTER_KEY` lives in a `.env` file; production needs a KMS or HSM. There is no HTTPS, login rate limiting or virus scanning, and registration reveals whether an e-mail is already used. For this scenario two design tensions remain: mandatory KTP verification protects accused persons against false reports but may deter victims who fear exposure, and a reporter's permission applies to all officers rather than only the assigned handler; the plaintext category linked to a reporter also reveals the type of violence. Out of scope: a compromised server or reporter device. In conclusion, Lapor Aman lets victims hand over evidence of violence so that it is encrypted at rest, released only to its owner, shown to officers only with consent and an audit trail, and verified to be intact on every read; AES-256-GCM provides this at negligible cost, while DES and RC4 add delay and, in DES's case, a weaker copy.

AI usage. ChatGPT: https://chatgpt.com/share/6ae05665-e3d4-83ec-832e-7b93d76c83d1 — *[TODO: what the group used ChatGPT for].* Claude Code (Anthropic) was used to apply the Huawei-style design and the violence scenario to the frontend, fix two frontend bugs, and draft this report from the code and the team's benchmark results; log: *[TODO: shared link to the Claude log]. [TODO: How the group checked the AI output, e.g. compared each claim with the code, re-ran tests and benchmark, checked the cited regulations.]*